

C++프로그래밍 및 실습 5주차 tic_tac_toe.pdf

인공지능학부 245131 장세인

① 서론

- (1) 프로젝트 목적 및 배경: 4주차까지 배운 내용에 대한 실습을 위해 진행
- (2) 목표 : Tic Tac Toe 게임 구현

② 요구사항

- (1) 사용자 요구사항: 두 명의 사용자가 번갈아가며 O와 X를 놓기
- (2) 기능 요구사항
 - 누구의 차례인지 출력
 - 좌표 입력 받기
 - 입력 받은 좌표 유효성 체크
 - 좌표에 O / X 놓기
 - 현재 보드판 출력
 - 빙고 시 승자 출력 후 종료
 - 모든 칸이 찼으면 종료
 - 단, 보드판은 2차원 배열 사용

③ 설계 및 구현

(1) 2차원 배열을 사용하여 보드판 생성 후 초기화

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      const int numCell = 3;
6      char board[numCell][numCell]{}; // 2차원 배열로 게임판 만들기
7      int x, y; // 사용자가 입력하는 좌표
8
9      // 보드판 빈칸으로 초기화
10     for (x = 0; x < numCell; x++) {
11         for (y = 0; y < numCell; y++) {
12             board[x][y] = ' ';
13         }
14     }
```

- const를 통하여 배열 크기가 될 3을 상수 선언
- 6번째 줄을 통해 3x3 배열 생성과 동시에 0이나 null로 초기화 시킴
- 사용자 입력 좌표가 들어갈 x, y 변수 선언
- 반복문을 통하여 0이나 null이 들어있던 2차원 배열의 모든 칸을 빈칸으로 초기화

(2) 누구의 차례인지 출력

```
16 // 게임 코드
17 int k = 0; // 몇 번째 차례인지
18 char currentUser = 'X'; // 현재 플레이어 저장을 위한 변수
19
20 // 종료 전까지 무한반복
21 while (true) {
22
23     // 1. 누구의 차례인지 출력
24     switch (k % 2) {
25     case 0:
26         cout << "1번 유저(X)의 차례입니다 -> ";
27         currentUser = 'X';
28         break;
29
30     case 1:
31         cout << "2번 유저(O)의 차례입니다 -> ";
32         currentUser = 'O';
33         break;
34     }
```

- 몇 번째 차례인지 계산할 k 변수 선언. 코드 끝에 k++을 통해 증가시키며 차례를 변경할 예정
- k%2를 통해 두 명의 플레이어가 각각 0과 1의 값을 할당 받을 예정이므로 switch, case 문을 통해 case 0과 case 1로 구분함. 이때 나머지 0을 할당 받은 유저는 1번 유저(X 기호 사용), 나머지 1을 할당 받은 유저는 2번 유저(O 기호 사용).
- break를 통하여 각각의 경우를 구분함.

(3) 좌표 입력 받기

```
36 // 2. 좌표 입력받기
37 cout << "(x, y) 좌표를 입력하세요: ";
38 cin >> x >> y;
39
```

- cout로 입력 지시 문구 출력 후, cin으로 x와 y 변수에 사용자 입력 값을 넣음

(4) 입력 받은 좌표의 유효성 체크

```
40 // 3. 입력받은 좌표의 유효성 체크
41 // 범위를 벗어난 경우
42 if (x < 0 || x >= numCell || y < 0 || y >= numCell) {
43     cout << "x와 y 중 하나가 판을 벗어났습니다." << endl;
44     continue; // k++이 continue보다 뒤에 위치하고 있기 때문에 같은 사람의 차례로 반복문의 처음으로 되돌아감
45 }
46
47 // 이미 돌이 놓여 있는 경우
48 if (board[x][y] != ' ') {
49     cout << x << ", " << y << " : 이미 돌이 차있습니다." << endl;
50     continue; // k++이 continue보다 뒤에 위치하고 있기 때문에 같은 사람의 차례로 반복문의 처음으로 되돌아감
51 }
```

- 사용자에게 입력 받은 좌표가 3x3 크기를 벗어났을 경우(0~2가 아닌 수), 혹은 이미 돌이 놓여있는 곳을 재입력 했을 경우 문구 출력과 함께 재입력 받음.
- continue보다 사용자의 차례를 증가시키는 k++이 뒤에 위치하고 있기 때문에, 유효성 체크에 걸리더라도 같은 사용자에게 값을 다시 입력 받을 수 있음.

(5) 좌표에 O / X 놓기

```
53 // 4. 좌표에 현재 유저의 돌 놓기
54 board[x][y] = currentUser;
```

- 사용자가 입력한 좌표에 해당 유저의 기호(1번 유저는 X, 2번 유저는 O)를 각각 대입함.

(6) 빙고 시 승자 출력 후 종료 (각각의 주석 확인)

1) 가로 확인

```
75 // 6. 빙고 확인
76 bool win = false; // false로 초기화하고 시작 후 win(true)이 되면 종료
77
78 // 가로 확인
79 for (int i = 0; i < numCell; i++) { // &&를 통해 중간에 조건이 거짓이면 중간에 종료됨 (단락평가)
80     if (board[i][0] != ' ' && // 가로줄 전부 빈칸일 경우 빙고로 판정되는 것 방지
81         board[i][0] == board[i][1] && // 각각 조건 만족 판정 후 모든 결과를 만족하는가(&&)
82         board[i][1] == board[i][2]) {
83         win = true;
84     }
85 }
```

2) 세로 확인

```
87 // 세로 확인(가로와 동일한 방법)
88 for (int j = 0; j < numCell; j++) {
89     if (board[0][j] != ' ' &&
90         board[0][j] == board[1][j] &&
91         board[1][j] == board[2][j]) {
92         win = true;
93     }
94 }
```

3) 대각선(우하향) 확인

```
96 // 우하향 대각선 확인 (대각선 좌표는 정해져있음 -> 반복문 필요 x)
97 if (board[0][0] != ' ' && // &&를 통해 중간에 조건이 거짓이면 중간에 종료됨 (단락평가)
98     board[0][0] == board[1][1] &&
99     board[1][1] == board[2][2]) {
100     win = true;
101 }
```

4) 대각선(좌하향) 확인

```
103 // 좌하향 대각선 확인 (대각선 좌표는 정해져있음 -> 반복문 필요 x)
104 if (board[0][2] != ' ' &&
105     board[0][2] == board[1][1] &&
106     board[1][1] == board[2][0]) {
107     win = true;
108 }
```

5) 모든 조건문 확인 후 빙고 시 승자 출력

```
110 // 빙고가 만들어졌으면 종료
111 if (win) {
112     cout << currentUser << " 유저가 승리했습니다!" << endl;
113     break;
114 }
```

(6) 모든 칸이 찼으면 종료

```
116 // 7. 모든 칸이 찼는지 확인 (칸이 다 찼는데 빙고가 만들어지지 않았을 경우)
117 bool full = true;
118
119 for (int i = 0; i < numCell; i++) { // 행 순회
120     for (int j = 0; j < numCell; j++) { // 열 순회
121         if (board[i][j] == ' ') { // 빈칸이 존재할 경우
122             full = false; // full을 false로 바꿈
123         }
124     }
125 }
126
127 if (full) { // 위의 반복문을 거쳤는데도 true로 유지될 경우
128     cout << "모든 칸이 찼습니다. 무승부입니다!" << endl;
129     break;
130 }
```

- 모든 칸을 반복문을 통해 순회한 후, 모든 칸이 채워졌다면 무승부 문구를 출력함.

④ 테스트

(1) 기능별 테스트 결과

```
2번 유저(o)의 차례입니다 -> (x, y) 좌표를 입력하세요: []
```

- 누구의 차례인지 출력

```
1번 유저(x)의 차례입니다 -> (x, y) 좌표를 입력하세요: 0 0
```

- 좌표 입력 받기

```
2번 유저(o)의 차례입니다 -> (x, y) 좌표를 입력하세요: 5 0
x와 y 중 하나가 판을 벗어났습니다.
2번 유저(o)의 차례입니다 -> (x, y) 좌표를 입력하세요: -1 -3
x와 y 중 하나가 판을 벗어났습니다.
2번 유저(o)의 차례입니다 -> (x, y) 좌표를 입력하세요: 0 0
0, 0 : 이미 돌이 차있습니다.
```

- 입력 받은 좌표 유효성 체크

```
---|---|---
x  |  |  |
---|---|---
  |  |  |
---|---|---
  |  |  |
---|---|---
```

- 좌표에 O / X 놓기, 현재 보드판 출력

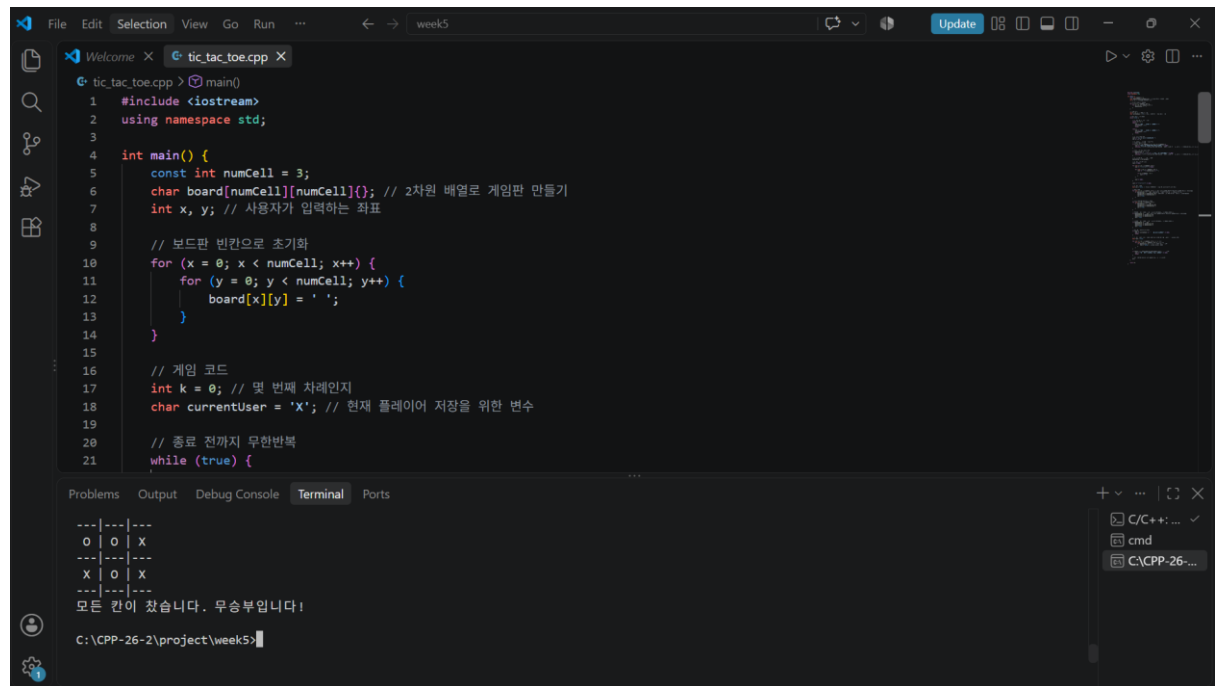
```
1번 유저(x)의 차례입니다 -> (x, y) 좌표를 입력하세요: 2 2
---|---|---
x  |  |  |
---|---|---
o  | x |  |
---|---|---
  | o | x |
---|---|---
x 유저가 승리했습니다!
```

- 빙고 시 승자 출력 후 종료

```
---|---|---
x  | x | o
---|---|---
o  | o | x
---|---|---
x  | o | x
---|---|---
모든 칸이 찼습니다. 무승부입니다!
```

- 모든 칸이 찼으면 무승부

(2) 프로그램 전체 스크린샷



⑤ 결과 및 결론

- (1) 프로젝트 결과: Tic Tac Toe 게임을 만들었음
- (2) 느낀점: 다른 언어들에서도 많이 접해본 코드였지만 c++문법으로 재작성 하려고 하니 까다로웠다.